DSA · RECURSION

Introduction to Recursion

Loops vs recursion, the recursive stack and the recursive tree

Today’s Agenda

1. Introduction to Recursion
2. Loops vs Recursion
3. Print 1 to n using Recursion
4. Recursive Stack
5. Recursive Tree
6. Practice Problems

INTRODUCTION TO RECURSION

The process in which a function calls itself, directly or indirectly, is called recursion.

The corresponding function is called a recursive function.

Print 1 to n using a loop

```
n = 5
i = 1

while i⩽n:
  print(i, end=" ")
  i+=1
```

Output

1 2 3 4 5

The loop repeats until i is greater than n.

Print 1 to n using recursion

```
def printNumbers(i, n):
  # base case
  if i>n:
    return
  # recursive case
  print(i, end=" ")
  printNumbers(i+1, n)

printNumbers(1, 5)
```

Output

```
1 2 3 4 5
```

Base case: stop when i > n.

Recursive case: print i, then call again with i + 1.

Call chain of printNumbers(1, 5)

Call 1	Call 2	Call 3	Call 4	Call 5	Call 6
`(1, 5)`	`(2, 5)`	`(3, 5)`	`(4, 5)`	`(5, 5)`	`(6, 5)`
prints 1	**prints 2**	**prints 3**	**prints 4**	**prints 5**	**returns**

Each call prints i, then calls itself with i + 1. When i is greater than n, the base case returns and the output is **1 2 3 4 5**.

Factorial using recursion

```
def fact(n):
  if n==0:
    return 1
  return n*fact(n-1)

print(fact(4))
```

Base case: n == 0 returns 1.

Recursive case: n times fact(n – 1).

fact(4) returns

24

Recursive stack: print before the call

```
def fun(n):
  if n==0:
    return
  print(n, end=" ")
  fun(n-1)

fun(3)
```

Output

```
3 2 1
```

The print runs before the recursive call, so each value is printed on the way down.

Recursive stack: print after the call

```
def fun(n):
  if n==0:
    return
  fun(n-1)
  print(n, end=" ")

fun(3)
```

Calls go down

- `fun(3)`
- `fun(2)`
- `fun(1)`
- `fun(0) returns`

Stack unwinds

- `print(1)`
- `print(2)`
- `print(3)`

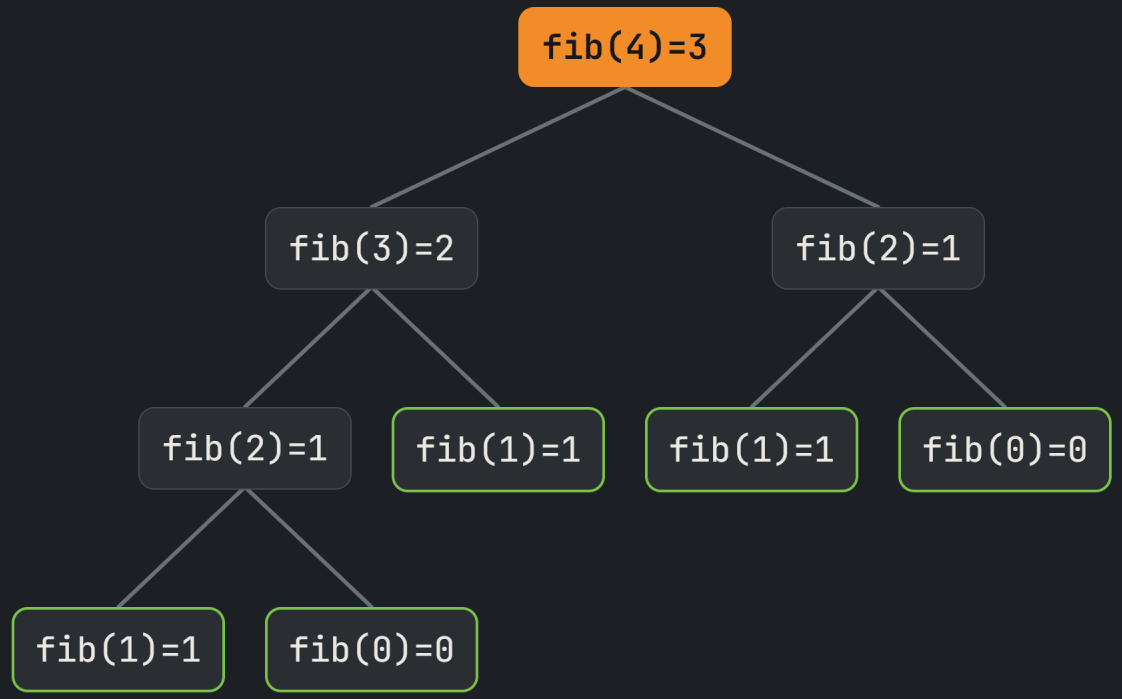
Output: **1 2 3**. The prints wait on the stack and run in reverse order once fun(0) returns.

Recursive tree: Fibonacci

```
def fib(self, n: int) → int:
    # base case
    if n==0 or n==1:
        return n
    # recursive case
    return fib(n-1) + fib(n-2)
```

fib(4) = **3**

Green outlines mark the base cases, n = 0 and n = 1.



Practice problems

For every problem, answer these three questions first.

1

What is the base case?

2

What is the recursive case?

3

What do the stack and tree look like for a small input?

[Add the practice problems from your session here]